



MODULE 07/09

Expo Router

Navigation file-based pour apps Expo

■ Durée : 1 semaine

■ Projet : Social Seller

■ Niveau : Débutant →
Opérationnel

Objectifs de la semaine

- ✓ Comprendre le routing file-based : la structure des dossiers = la navigation
- ✓ Créer des layouts partagés avec `_layout.tsx`
- ✓ Naviguer avec `useRouter` et `Link` entre les écrans
- ✓ Utiliser les routes dynamiques `[id].tsx` pour les détails
- ✓ Configurer les onglets, les deep links et le scheme URI

Planning de la semaine

JOUR 1	Structure de fichiers	2-3h
	• <code>app/</code> = racine de la navigation • <code>_layout.tsx</code> : layout partagé pour un groupe de routes • <code>index.tsx</code> : route par défaut d'un dossier • <code>+not-found.tsx</code> : page 404 personnalisée	
JOUR 2	Navigation et hooks	2-3h
	• <code>useRouter()</code> : <code>push</code>, <code>replace</code>, <code>back</code>, <code>navigate</code> • <code>useLocalSearchParams()</code> : lire les paramètres d'URL • <code>Link</code> : naviguer de façon déclarative dans le JSX • <code>Redirect</code> : rediriger programmatiquement	
JOUR 3	Layouts et groupes	2-3h
	• <code>(auth)/</code> : groupe sans impact sur l'URL, layout partagé • <code>(tabs)/</code> : layout avec onglets en bas • <code>Stack.Screen</code> options : titre, bouton retour, header custom • <code>Tabs.Screen</code> <code>tabBarIcon</code>, <code>tabBarLabel</code>	
JOUR 4	Deep links et scheme	2-3h
	• <code>scheme</code>: dans <code>app.json</code> — ex: <code>socialseller://</code> • Deep link vers une route : <code>socialseller://conversation/abc-123</code> • <code>Linking.getInitialURL()</code> — gérer l'ouverture via lien • <code>expo-web-browser</code> <code>openAuthSessionAsync</code> pour OAuth	
JOUR 5	Mini-projet de la semaine	3-4h
	• Recréer la structure de navigation de Social Seller : • <code>(auth)/</code> : <code>login.tsx</code>, <code>register.tsx</code>	

- (tabs)/ : index.tsx, inbox.tsx, orders.tsx, stock.tsx, stats.tsx
- conversation/[id].tsx : route dynamique
- Ajouter les icônes Feather sur chaque onglet

Concepts clés détaillés

Structure de fichiers = Navigation

```
app/  
  ■■■ _layout.tsx → Layout racine (providers globaux)  
  ■■■ (auth)/ → Groupe auth (pas dans l'URL)  
  ■ ■■■ _layout.tsx → Layout du groupe (ex: pas de tabs)  
  ■ ■■■ login.tsx → Route /login  
  ■ ■■■ register.tsx → Route /register  
  ■■■ (tabs)/ → Groupe avec onglets en bas  
  ■ ■■■ _layout.tsx → Définit les onglets  
  ■ ■■■ index.tsx → Onglet Dashboard (route /)  
  ■ ■■■ inbox.tsx → Onglet Inbox (route /inbox)  
  ■ ■■■ stats.tsx → Onglet Stats (route /stats)  
  ■■■ conversation/  
  ■ ■■■ [id].tsx → Route /conversation/abc-123  
  ■■■ product/  
  ■ ■■■ [id].tsx → Route /product/xyz-456  
  ■ ■■■ new.tsx → Route /product/new  
  ■■■ +not-found.tsx → 404
```

_layout.tsx des onglets

```
// app/(tabs)/_layout.tsx  
import { Tabs } from 'expo-router';  
import { Feather } from '@expo/vector-icons';  
  
export default function TabsLayout() {  
  return (  
    <Tabs screenOptions={{  
      tabBarActiveTintColor: '#6366F1',  
      tabBarInactiveTintColor: '#94A3B8',  
      headerShown: false,
```

```

}}>
<Tabs.Screen
name='index'
options={{
  tabBarLabel: 'Dashboard',
  tabBarIcon: ({ color }) => (
    <Feather name='home' size={22} color={color} />
  ),
}}
/>
<Tabs.Screen
name='inbox'
options={{
  tabBarLabel: 'Inbox',
  tabBarIcon: ({ color }) => (
    <Feather name='inbox' size={22} color={color} />
  ),
}}
/>
</Tabs>
);
}

```

Navigation programmatique

```

import { useRouter, useLocalSearchParams } from 'expo-router';

// Dans un écran liste → naviguer vers le détail
export default function InboxScreen() {
  const router = useRouter();

  return (
    <FlatList
      renderItem={({ item }) => (
        <Pressable onPress={() => router.push(`/conversation/${item.id}`)}>
          <Text>{item.customer_name}</Text>
        </Pressable>
      )}
    />
  );
}

```

```

);
}

// Dans l'écran détail → récupérer l'id
export default function ConversationScreen() {
  const router = useRouter();
  const { id } = useLocalSearchParams<{ id: string }>();

  return (
    <Pressable onPress={() => router.back()}>
      <Feather name='arrow-left' size={20} />
    </Pressable>
  );
}

```

Deep links et OAuth (cas réel Social Seller)

```

// app.json – définir le scheme URI
{ "expo": { "scheme": "socialseller" } }

// Résultat : socialseller://oauth-success?platform=whatsapp

// Dans channels.tsx – ouvrir le flow OAuth
import * as WebBrowser from 'expo-web-browser';

const result = await WebBrowser.openAuthSessionAsync(
  authorizationUrl,
  'socialseller://' // Scheme de rappel – iOS ASWebAuthenticationSession
);

if (result.type === 'success') {
  // result.url = 'socialseller://oauth-success?platform=whatsapp'
  // Refetch les connexions
  await fetchConnections();
}

```

■ Expo Router est basé sur React Navigation. Le routing file-based s'inspire de Next.js. Un fichier = une route. Un dossier avec `_layout.tsx` = un groupe avec layout partagé. Les parenthèses dans les noms de dossiers (`auth`), (`tabs`) = groupes qui n'apparaissent pas dans l'URL.

Exercices pratiques

Exercice 1 — Structure de navigation (30 min)

- Créer une app Expo avec la structure : (auth)/login.tsx, (tabs)/index.tsx, (tabs)/profile.tsx
- Le layout racine redirige vers /login si non connecté, sinon vers les tabs
- Le layout tabs affiche 2 onglets avec icônes

Exercice 2 — Route dynamique (45 min)

- Créer un écran liste de produits sur (tabs)/products.tsx
- Créer product/[id].tsx qui affiche le détail du produit sélectionné
- Passer le nom du produit via l'URL et l'afficher dans le header

Exercice 3 — Protection de routes (30 min)

- Créer un contexte d'authentification avec useState
 - Dans _layout.tsx racine, utiliser Redirect vers /login si non connecté
 - Tester que naviguer vers /tabs/index sans être connecté redirige vers /login
-

Ressources pour aller plus loin

- Expo Router docs : docs.expo.dev/router/introduction
- Expo Router exemples : github.com/expo/expo/tree/main/templates
- React Navigation (base d'Expo Router) : reactnavigation.org